

Coerência de Cache e Falso Compartilhamento

Eugenio Vitor Lopes dos Santos

DCA3703, Programação Paralela, Universidade Federal do Rio Grande do Norte

1 Enunciado

Implementar em C a estimativa estocástica de π usando `rand()` para gerar os pontos. Cada thread deve usar uma variável privada para contar os acertos e acumular o total em uma variável global com `#pragma omp critical`. Depois, implementar uma segunda versão em que cada thread escreve seus acertos em uma posição distinta de um vetor compartilhado. A acumulação deve ser feita em um laço serial após a região paralela. Comparar o tempo de execução das duas versões. Em seguida, substituir `rand()` por `rand_r()` em ambas e comparar novamente. Explicar o comportamento dos quatro programas com base na coerência de cache e nos efeitos do falso compartilhamento.

2 Fundamentação teórica

2.1 Método de Monte Carlo

O quadrado $[-1, 1] \times [-1, 1]$ tem área 4, e o círculo unitário inscrito tem área π . Sorteando pontos uniformes no quadrado e testando $x^2 + y^2 \leq 1$, a fração de acertos converge para a razão das áreas: $n_{dentro}/N \approx \pi/4$. A estimativa é $\hat{\pi} = 4 \cdot n_{dentro}/N$. O erro cai com $1/\sqrt{N}$, e o gerador define tanto a qualidade estatística quanto o custo computacional.

2.2 `rand` com estado global contra `rand_r` com estado privado

O `rand()` guarda o estado em variável global oculta na libc. Cada chamada lê, atualiza e escreve esse estado. Com várias threads, sobram duas saídas ruins. Sem proteção, ocorre *data race* e a sequência se corrompe. Com `critical`, a geração serializa e só uma thread gera por vez. A função `rand_r(unsigned *seed)` recebe o estado por parâmetro. Se cada thread mantém `unsigned seed` local privada, cada uma avança sua própria sequência sem comunicação, e a geração paraleliza de verdade. Vetor de sementes contíguo `seeds[8]` reintroduz o problema: as sementes dividem a mesma linha de cache, e cada `rand_r` invalida a linha dos vizinhos.

2.3 `critical` por ponto contra privado mais redução serial

A versão A usa contador privado `local` e uma única entrada em `critical` por thread: `total += local`. O custo de sincronização é de P entradas, e o laço não tem tráfego coerente. A versão B com vetor escreve `hits[tid]++` no laço e soma em laço serial após a barreira. Se a escrita no vetor ocorre a cada acerto, há N escritas concorrentes em posições vizinhas. A forma correta é manter a contagem privada durante o laço e realizar uma única escrita em `hits[tid]` no epílogo, seguida de soma serial de P elementos.

2.4 Coerência de cache com caches privadas

Cada núcleo só enxerga a RAM através de suas caches privadas L1/L2. Para manter a ilusão de memória única, o hardware controla a coerência por linha de 64 B, com estados MESI e dois bits por linha: sujo e inválido. Em *write-back*, o store escreve só na cache e marca *dirty* = 1. O hardware só escreve na RAM na evicção ou quando outro núcleo pede a linha. Em *write-through*, cada store atravessa até a memória. Quando um núcleo escreve em linha compartilhada, o hardware invalida as cópias nos vizinhos. O próximo load sofre miss e busca o valor novo. Uma variável compartilhada custa pouco para leitura ou escrita rara: basta copiar com `private` e `firstprivate`. Para escrita frequente, ela torna a cache inútil. Sem sincronização, há condição de corrida. Com `critical`, há serialização. Sem nenhum dos dois, há ping-pong de invalidações. No pior caso, o miss em linha suja remota custa dois acessos: o dono escreve a linha de volta, e o solicitante a busca em seguida.

2.5 Falso compartilhamento

A unidade de coerência é a linha, não a variável. Um vetor `long hits[8]` ocupa 64 B e preenche exatamente uma linha de cache: oito contadores distintos disputam a mesma linha. Escritas em variáveis logicamente independentes invalidam-se mutuamente, e a linha pinga entre caches a cada iteração, mesmo sem compartilhamento lógico. A correção é afastar cada posição em 64 B distintos com *padding*: `struct { long v; char pad[56]; } __attribute__((aligned(64)))`. Declarar `volatile`, trocar *write-back* por *write-through* ou envolver em `critical` não resolve: o primeiro não muda o layout, o segundo piora o tráfego, e o terceiro serializa.

2.6 Snooping contra diretório

Em *snooping*, o hardware difunde cada miss ou escrita no barramento, e todas as caches espiam para ver se têm a linha. É *broadcast* contínuo e não escala com dezenas de núcleos. No protocolo com diretório, um registro central junto à LLC guarda quem tem cada linha, e o hardware envia invalidação só a esses donos. O falso compartilhamento gera avisos inúteis nos dois casos, mas no *snooping* o custo interrompe todos os nós.

3 Implementação

Foram avaliadas cinco versões em C11 com medição via `clock_gettime(CLOCK_MONOTONIC)` e controle de escopo com `default(none)`: v1 `critical+rand`, v2 `vetor+rand`, v3 `critical+rand_r`, v4 `vetor+rand_r` e v4-pad com *slot* de 64 B. Em v1 e v3, cada thread conta em `long local` e faz um único `#pragma omp critical total += local`. Em v2 e v4, cada thread incrementa `hits[tid]` no laço quente para expor o falso compartilhamento, e a soma é serial após a região paralela. Em v3, v4 e v4-pad, a semente é local: `unsigned seed = 123456789U ^ tid`. Compilação com GCC 13.3, `-std=c11 -O2 -Wall -fopenmp -lm`, $N = 5.000.000$.

```
1 gcc -std=c11 -O2 -Wall -fopenmp v1_pi_critical_rand.c -o pi_critical_rand -lm
2 gcc -std=c11 -O2 -Wall -fopenmp v2_pi_vector_rand.c -o pi_vector_rand -lm
```

```

3 gcc -std=c11 -O2 -Wall -fopenmp v3_pi_critical_randr.c -o
   pi_critical_randr -lm
4 gcc -std=c11 -O2 -Wall -fopenmp v4_pi_vector_randr.c -o pi_vector_randr -
   lm
5 gcc -std=c11 -O2 -Wall -fopenmp v4_fix_pi_vector_randr_pad.c -o
   pi_vector_randr_pad -lm
6 OMP_NUM_THREADS=8 ./pi_critical_randr 5000000
7 OMP_NUM_THREADS=8 ./pi_vector_randr_pad 5000000

```

Listing 1: Compilação e execução

4 Resultados e discussão

Testes em Intel Core i7-14700HX sob WSL2 com 14 núcleos físicos e 28 threads lógicas, varredura de 1 a 28 threads com $N = 5.000.000$. Cada ponto é a mediana de cinco repetições. As Figuras 1 e 2 mostram a curva completa em escala linear e em escala logarítmica, e a Figura 3 compara as cinco versões com 8 threads. O CSV completo com 28 linhas acompanha o relatório.

Table 1: Tempo em segundos com $N = 5.000.000$, varredura de 1 a 28 threads. Cada valor é a mediana de cinco repetições.

Thr	v1 crit+rand	v2 vet+rand	v3 crit+rand_r	v4 vet+rand_r	v4-pad
1	0,2010	0,2039	0,0514	0,0797	0,0802
2	0,9143	1,0549	0,0268	0,0491	0,0413
3	0,7179	0,6643	0,0160	0,0276	0,0231
4	0,6124	0,7180	0,0137	0,0253	0,0207
5	0,7818	0,8776	0,0097	0,0194	0,0139
6	0,9890	1,0890	0,0096	0,0238	0,0137
7	1,1248	1,2938	0,0096	0,0213	0,0130
8	1,2765	1,3067	0,0084	0,0248	0,0118
9	1,2250	1,3819	0,0076	0,0240	0,0111
10	1,3017	1,4150	0,0070	0,0308	0,0108
11	1,2607	1,2944	0,0063	0,0251	0,0098
12	1,2711	1,4511	0,0059	0,0240	0,0088
13	1,3825	1,4457	0,0058	0,0276	0,0088
14	1,3487	1,4307	0,0052	0,0258	0,0086
15	1,3305	1,4581	0,0054	0,0271	0,0079
16	1,3362	1,5018	0,0052	0,0280	0,0075
17	1,3698	1,4792	0,0046	0,0260	0,0072
18	1,3805	1,4689	0,0045	0,0296	0,0073
19	1,3678	1,4139	0,0153	0,0289	0,0067
20	1,4117	1,4441	0,0046	0,0272	0,0064
21	1,3862	1,2353	0,0299	0,0404	0,0192
22	1,4617	1,3128	0,0321	0,0395	0,0193
23	1,1848	1,2005	0,0323	0,0465	0,0243
24	1,2333	1,2932	0,0278	0,0489	0,0203
25	1,1760	1,2269	0,0376	0,0471	0,0267
26	1,2063	1,2430	0,0355	0,0461	0,0281
27	1,3071	1,2105	0,0370	0,0503	0,0276
28	1,4518	1,3675	0,0302	0,0398	0,0266

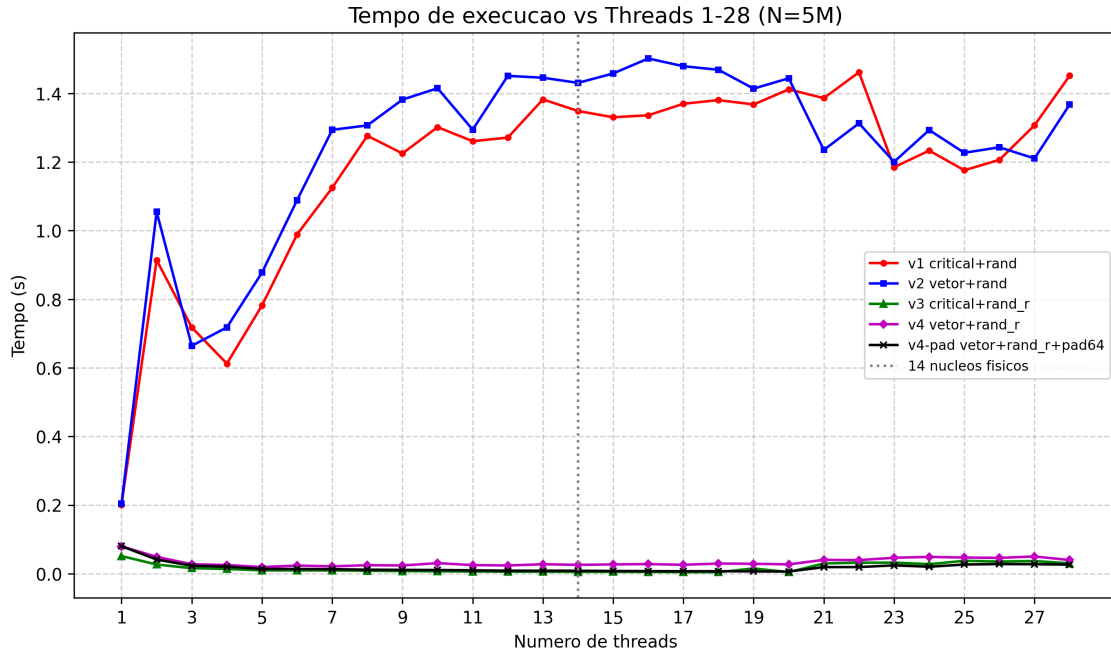


Figure 1: Tempo contra threads de 1 a 28 em escala linear. As curvas com *rand* sobem com o número de threads; com *rand_r*, descem até 14 threads e oscilam depois, com a linha tracejada nos 14 núcleos físicos.

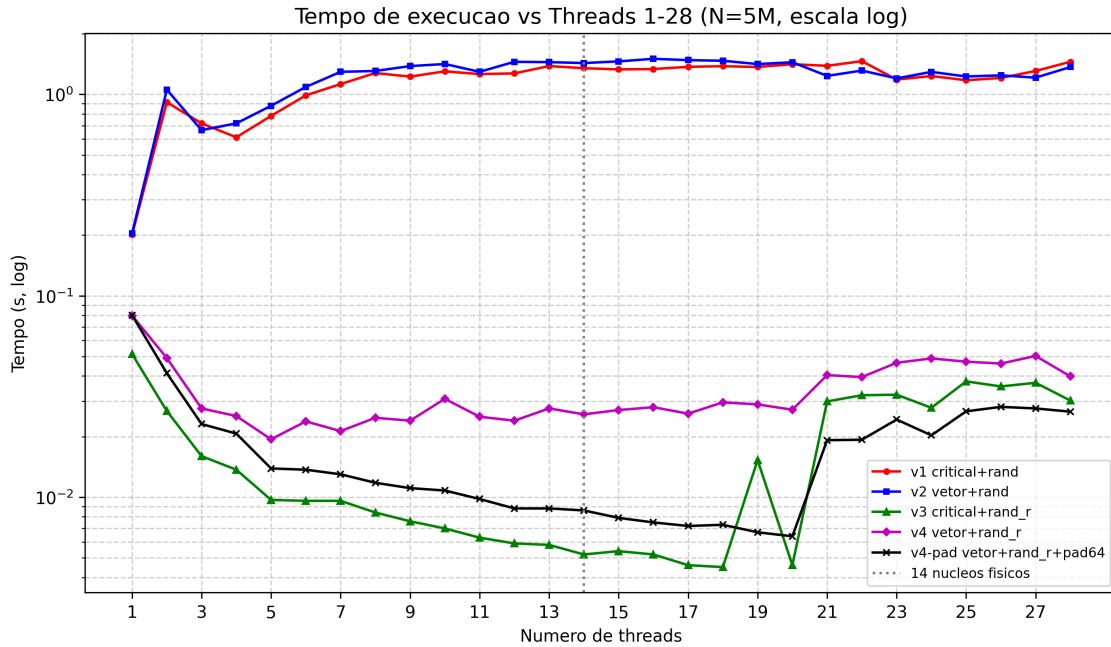


Figure 2: Os mesmos dados em escala logarítmica no eixo y . O ganho da versão v3 sobre v1 e v2 fica visível em duas ordens de grandeza, e a separação entre v4 sem e com *padding* aparece em toda a faixa.

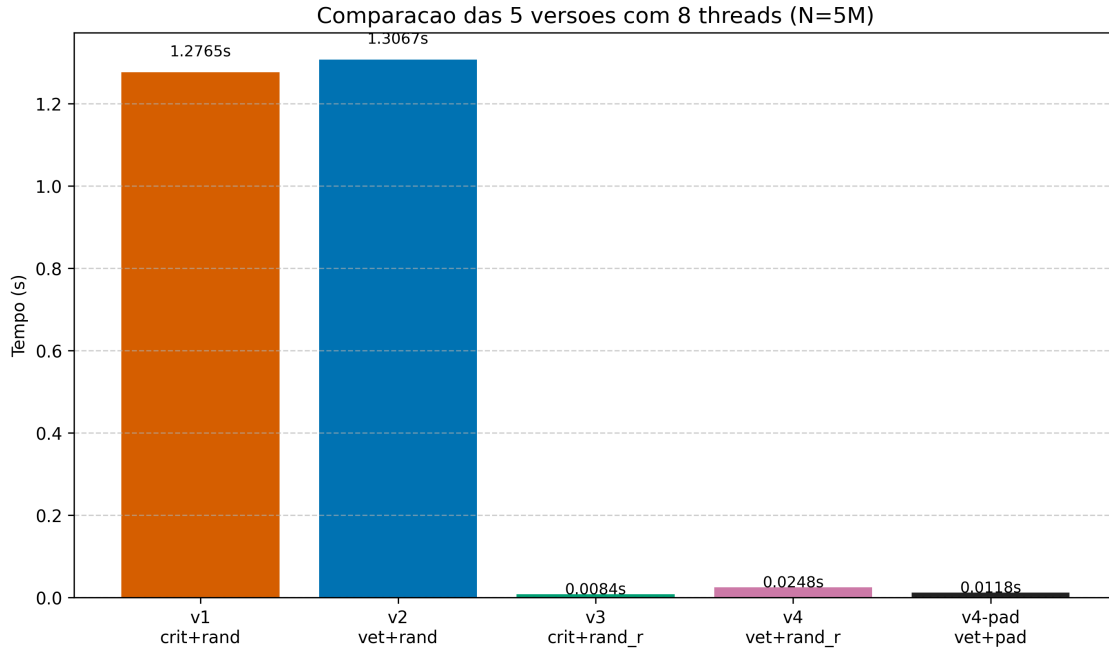


Figure 3: Comparação com 8 threads: as versões v1 e v2 são duas ordens de grandeza mais lentas que a v3.

4.1 Por que v1 e v2 não escalam

Com 1 thread, v1 e v2 levam cerca de 0,20 s. Com 8 threads, passam de 1,3 s: adicionar threads deixa o programa mais lento. A função `rand()` tem trava interna e estado global. Todas as threads disputam a mesma linha de cache, e a geração serializa. Mais threads acarretam maior contenção e mais tráfego de coerência no protocolo MESI. O *speedup* fica bem abaixo de 1. A diferença entre v1 e v2 é pequena porque o gargalo está no gerador, não na soma: trocar `critical` final por vetor compartilhado não destrava a geração. Além disso, os valores estimados de π variam entre execuções com mais de uma thread, sintoma evidente de *data race* no estado de `rand`.

4.2 Por que v3 é a mais rápida

O tempo de v3 cai de 0,0514 s com 1 thread para 0,0045 s com 18 threads, alcançando um ganho de 11,4 $\times$. Até 20 threads, o tempo permanece entre 0,004 s e 0,008 s. Acima de 21 threads, sob efeito de SMT e da virtualização WSL2, sobe para a faixa de 0,027 s a 0,037 s. Cada thread avança sua semente local sem trava e sem tráfego de coerência, e a soma custa apenas P entradas na seção crítica. É o padrão ótimo de projeto: acúmulo privado durante o laço e sincronização única no epílogo. A ordem de desempenho prevista ($1 > 2 > 3$) confirma-se, e o salto $1 \rightarrow 3$ é o mais expressivo por eliminar a serialização das N gerações.

4.3 Falso compartilhamento visível em v4

A versão v4 é cerca de 2 a 5 $\times$ mais lenta que a v3 na mesma contagem de threads: com 8 threads, consome 0,0248 s contra 0,0084 s; com 14 threads, 0,0258 s contra 0,0052 s. Sem a trava de `rand` para mascarar a contenção, o ping-pong das linhas de cache contendo o vetor `hits[tid]++` torna-se dominante. Os P contadores cabem em 1 ou 2 linhas de 64 B, e cada

acerto invalida a linha nos demais núcleos. A variante com *padding* de 64 B reduz o tempo para 0,0118 s com 8 threads e 0,0086 s com 14 threads (ganho de $2,1\times$ a $3,0\times$ sobre v4 sem *padding*), confirmando o diagnóstico. O resíduo em relação à v3 decorre do fato de as escritas frequentes ainda percorrerem a hierarquia de memória, mesmo sem invalidação cruzada.

5 Conclusão

O fator determinante para o tempo de execução é a geração de números pseudoaleatórios, e não a soma dos acertos. O uso de `rand()` com estado global inviabiliza a escalabilidade: as versões v1 e v2 desaceleram com mais threads, atingindo cerca de 1,3 s com 8 threads frente a 0,20 s com 1 thread. A substituição por `rand_r()` com semente privada destrava a geração e evidencia o custo da soma: v3, com contador privado e redução única ao final, é a mais rápida; v4, com vetor contíguo compartilhado, sofre severamente com falso compartilhamento, efeito mitigado pelo alinhamento e *padding* de 64 B. A diretriz prática para programação paralela é: estado estocástico estritamente privado por thread, acumulação privada ao longo do laço com redução única no final, e separação de dados compartilhados suscetíveis a escrita frequente em linhas de cache distintas.

A Código-fonte completo

```
1  /* v1: rand() + contador privado + critical (1x por thread)
2  * Compilar: gcc -O2 -fopenmp -Wall -o pi_critical_rand pi_critical_rand.c
   *      -lm
3  * Uso: OMP_NUM_THREADS=4 ./pi_critical_rand [total_elementos]
4  */
5  #define _POSIX_C_SOURCE 199309L
6  #include <stdio.h>
7  #include <stdlib.h>
8  #include <math.h>
9  #include <omp.h>
10 #include <time.h>
11
12 static double agora(void){
13     struct timespec t;
14     clock_gettime(CLOCK_MONOTONIC,&t);
15     return t.tv_sec + t.tv_nsec/1e9;
16 }
17
18 int main(int argc, char *argv[]){
19     long total_elementos = argc>1 ? atol(argv[1]) : 5000000L;
20     long total = 0;
21     double tempo = agora();
22
23     srand(12345);
24
25     #pragma omp parallel shared(total,total_elementos) default(none)
26     {
```

```

27     long local = 0;
28
29     #pragma omp for
30     for(long i = 0; i < total_elementos; i++){
31         double x = (double)rand()/((double)RAND_MAX * 2.0 - 1.0);
32         double y = (double)rand()/((double)RAND_MAX * 2.0 - 1.0);
33
34         if(x*x + y*y <= 1.0) {
35             local++;
36         }
37     }
38
39     #pragma omp critical
40     total += local;
41 }
42
43 tempo = agora() - tempo;
44
45 printf("v1 critical+rand: pi=%.6f tempo=%.4f s total=%ld\n", 4.0*total
46       /total_elementos, tempo, total);
47
48 return 0;
49 }

```

Listing 2: v1_pi_critical_rand.c

```

1  /* v2: rand() + vetor compartilhado hits[tid]++ (false sharing proposital)
2  * Compilar: gcc -O2 -fopenmp -Wall -o pi_vector_rand pi_vector_rand.c -lm
3  */
4  #define _POSIX_C_SOURCE 199309L
5  #include <stdio.h>
6  #include <stdlib.h>
7  #include <omp.h>
8  #include <time.h>
9
10 static double agora(void){
11     struct timespec t;
12     clock_gettime(CLOCK_MONOTONIC,&t);
13     return t.tv_sec + t.tv_nsec/1e9;
14 }
15
16 int main(int argc, char *argv[]){
17     long total_elementos = argc>1 ? atol(argv[1]) : 5000000L;
18     int total_threads = omp_get_max_threads();
19     long *hits = calloc(total_threads, sizeof(long));
20
21     srand(12345);
22
23     double tempo = agora();
24

```

```

25 #pragma omp parallel shared(hits,total_elementos) default(none)
26 {
27     int tid = omp_get_thread_num();
28
29     #pragma omp for
30     for(long i = 0; i < total_elementos; i++){
31         double x = (double)rand()/((double)RAND_MAX * 2.0 - 1.0);
32         double y = (double)rand()/((double)RAND_MAX * 2.0 - 1.0);
33
34         if(x*x + y*y <= 1.0) {
35             hits[tid]++;
36         }
37     }
38 }
39
40 long total = 0;
41
42 for(int i = 0; i < total_threads; i++) {
43     total += hits[i];
44 }
45
46 tempo = agora() - tempo;
47
48 printf("v2 vetor+rand: pi=%.6f tempo=%.4f s total=%ld\n", 4.0*total/
49     total_elementos, tempo, total);
50
51 free(hits);
52 return 0;
53 }

```

Listing 3: v2_pi_vector_rand.c

```

1  /* v3: rand_r() + contador privado + critical (1x por thread)
2   * Cada thread tem seed privada: elimina o gargalo global do rand().
3   */
4  #define _POSIX_C_SOURCE 199309L
5  #include <stdio.h>
6  #include <stdlib.h>
7  #include <omp.h>
8  #include <time.h>
9
10 static double agora(void){
11     struct timespec t;
12     clock_gettime(CLOCK_MONOTONIC,&t);
13     return t.tv_sec + t.tv_nsec/1e9;
14 }
15
16 int main(int argc, char *argv[]){
17     long total_elementos = argc>1 ? atol(argv[1]) : 5000000L;
18     long total = 0;

```

```

19     double tempo = agora();
20
21     #pragma omp parallel shared(total,total_elementos) default(none)
22     {
23         unsigned int seed = 123456789U ^ (unsigned int)omp_get_thread_num
24         ();
25         long local = 0;
26
27         #pragma omp for
28         for(long i = 0; i < total_elementos; i++){
29             double x = (double)rand_r(&seed)/(double)RAND_MAX * 2.0 - 1.0;
30             double y = (double)rand_r(&seed)/(double)RAND_MAX * 2.0 - 1.0;
31
32             if(x*x + y*y <= 1.0) {
33                 local++;
34             }
35
36             #pragma omp critical
37             total += local;
38         }
39
40         tempo = agora() - tempo;
41
42         printf("v3 critical+rand_r: pi=%.6f tempo=%.4f s total=%ld\n", 4.0*
43             total/total_elementos, tempo, total);
44
45         return 0;
46     }

```

Listing 4: v3_pi_critical_randr.c

```

1  /* v4: rand_r() + vetor compartilhado hits[tid]++ (false sharing visivel)
2   * Sem o lock do rand(), o ping-pong MESI do vetor aparece no tempo.
3   */
4  #define _POSIX_C_SOURCE 199309L
5  #include <stdio.h>
6  #include <stdlib.h>
7  #include <omp.h>
8  #include <time.h>
9  static double agora(void){
10     struct timespec t;
11     clock_gettime(CLOCK_MONOTONIC,&t);
12     return t.tv_sec + t.tv_nsec/1e9;
13 }
14
15 int main(int argc, char *argv[]){
16     long numero_elementos = argc>1 ? atol(argv[1]) : 5000000L;
17     int total_threads = omp_get_max_threads();
18

```

```

19     long *hits = calloc(total_threads, sizeof(long));
20
21     double tempo = agora();
22
23     #pragma omp parallel shared(hits, numero_elementos) default(none)
24     {
25         unsigned int seed = 123456789U ^ (unsigned int)omp_get_thread_num
                ();
26
27         int tid = omp_get_thread_num();
28
29         #pragma omp for
30         for(long i=0; i<numero_elementos; i++){
31             double x = (double)rand_r(&seed)/(double)RAND_MAX*2.0-1.0;
32             double y = (double)rand_r(&seed)/(double)RAND_MAX*2.0-1.0;
33             if(x*x+y*y<=1.0) hits[tid]++;
34         }
35     }
36     long total=0; for(int i=0; i<total_threads; i++) total+=hits[i];
37
38     tempo = agora() - tempo;
39
40     printf("v4 vetor+rand_r: pi=%.6f tempo=%.4f s total=%ld\n", 4.0*total/
            numero_elementos, tempo, total);
41
42     free(hits);
43
44     return 0;
45 }

```

Listing 5: v4_pi_vector_randr.c

```

1  /* v4 corrigida: padding de 64 bytes por posicao elimina false sharing.
2  * Cada slot tem 64 bytes (linha de cache): hits ficam em linhas distintas
3  */
4  #define _POSIX_C_SOURCE 199309L
5  #include <stdio.h>
6  #include <stdlib.h>
7  #include <omp.h>
8  #include <time.h>
9
10 typedef struct {
11     long v;
12     char pad[64-sizeof(long)];
13 } Slot __attribute__((aligned(64)));
14
15 static double agora(void){
16     struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t);
17     return t.tv_sec + t.tv_nsec/1e9;

```

```

18 }
19
20 int main(int argc, char *argv[]){
21     long total_elementos = argc>1 ? atol(argv[1]) : 5000000L;
22     int total_threads = omp_get_max_threads();
23
24     Slot *hits = calloc(total_threads, sizeof(Slot));
25
26     double tempo = agora();
27
28     #pragma omp parallel shared(hits,total_elementos) default(none)
29     {
30         unsigned int seed = 123456789U ^ (unsigned int)omp_get_thread_num
31         ();
32         int tid = omp_get_thread_num();
33
34         #pragma omp for
35         for(long i=0;i<total_elementos;i++){
36             double x = (double)rand_r(&seed) / (double)RAND_MAX * 2.0 -
37             1.0;
38             double y = (double)rand_r(&seed) / (double)RAND_MAX * 2.0 -
39             1.0;
40
41             if( x*x + y*y <= 1.0 ) hits[tid].v++;
42         }
43
44         long total = 0;
45         for(int i = 0; i < total_threads; i++) total += hits[i].v;
46
47         tempo = agora() - tempo;
48
49         printf("v4-pad vetor+rand_r+pad64: pi=%.6f tempo=%.4f s total=%ld\n",
50             4.0*total/total_elementos, tempo, total);
51
52         free(hits); return 0;
53     }
54 }

```

Listing 6: v4_fix_pi_vector_randr_pad.c